Upgrade Portal System

EMR Upgrade & New Shell Request Management Portal

Abdul Rauf Ibrahimkhail
MSIT 5910-01: Capstone Project
University of the People · May 2026

Introduction & Problem Statement

Why this portal was needed

Fragmented tracking

Upgrade schedules managed in Excel spreadsheets with no field validation, no status visibility, and no audit trail.

Incomplete requests

New environment requests arrive through Jira without required technical details, forcing TechOps to follow up before work can start.

Coordination overhead

Repeated clarification cycles between Professional Services and TechOps create delays that impact client go-live timelines.

Solution: A purpose-built web portal that enforces complete request intake, automates TechOps notifications, and gives administrators real-time status visibility across all active workflows.

Project Objectives & Scope

What we set out to achieve and the boundaries of the work

Objectives (SMART)

- Deliver a working prototype for upgrade scheduling and Shell Request intake
- Enforce mandatory field validation to eliminate incomplete submissions
- Implement SHA-256 hashing, 2FA, and RBAC from day one — not retrofitted
- Complete all phases within the 8-unit capstone timeline

In Scope

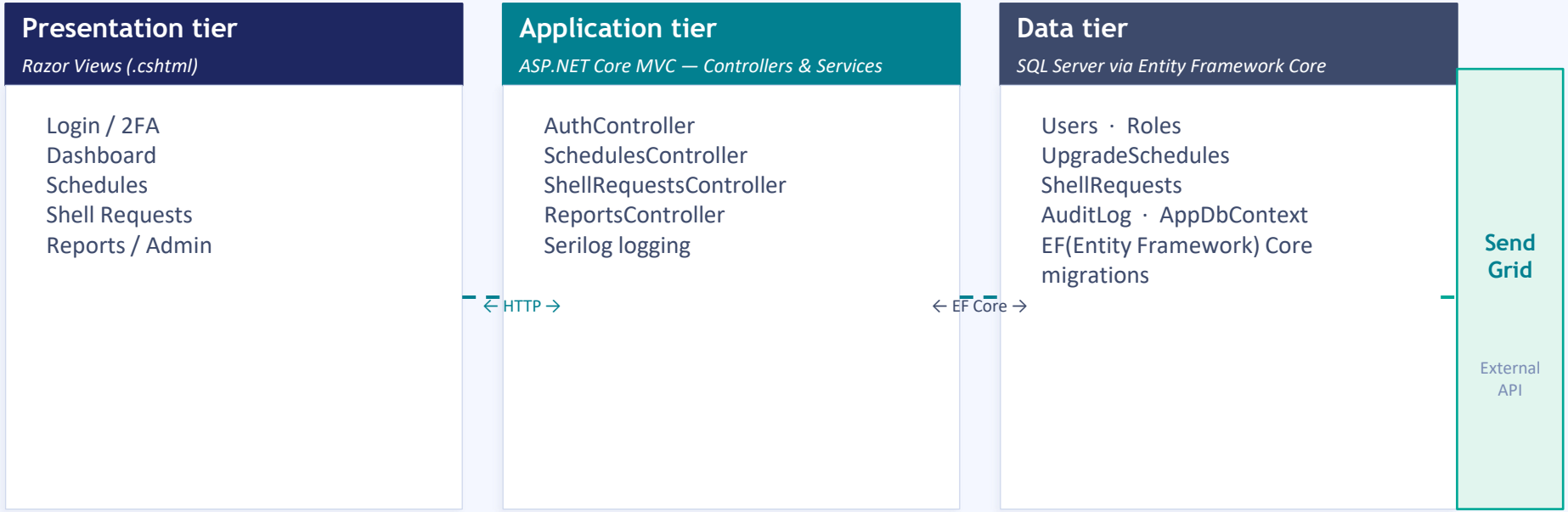
- Role-based login with 2FA
- Upgrade schedule management
- New Shell Request intake with SendGrid notifications
- CSV report export
- User administration & audit logging

Out of Scope

- Live EMR system integration
- Enterprise / production deployment
- Customer-facing external portal

System Architecture

Three-tier layered architecture with external SendGrid integration



Tech stack: ASP.NET Core MVC · SQL Server · Entity Framework Core · SendGrid · Serilog · GitHub

System Modules & Key Features

Six integrated modules covering the full operational workflow

01

Authentication

SHA-256 hashing · 2FA via SendGrid · Password reset · Session management

02

Schedule Mgmt

Create / update upgrades · Status tracking · CSV export · Audit trail

03

Shell Requests

Structured intake form · Mandatory field validation · Auto TechOps email

04

User Admin

Create users with roles · 2FA flag · Active/inactive toggle · RBAC

05

Reporting

Server-side CSV export · EF Core query · UTF-8 FileContentResult

06

Logging

Serilog daily rolling files · Login · DB writes · Email dispatch · Errors

Technologies, Frameworks & Methodology

Design decisions grounded in software engineering literature

Technology	Role in the Project
ASP.NET Core MVC	Web framework — MVC pattern separates concerns; native routing, DI, and validation
SQL Server + EF Core	Relational data + ORM — async Add/SaveChangesAsync pattern; explicit column mappings
SendGrid Web API	Email delivery — 2FA codes, password resets, TechOps shell request alerts
SHA-256 + Session	Security — credentials hashed at submission; session middleware for 2FA state
Serilog	Structured logging — daily rolling files; login, DB writes, email status captured
GitHub (2-branch)	Version control — main (stable) + development; v1.0.0–v1.0.4 semantic tags

Methodology: Prototype Development Model — 8 iterative capstone units, each extending tested foundations from the previous.

Core Algorithms & Design Decisions

Key implementation patterns with rationale

SHA-256 auth

```
var bytes =
sha256.ComputeHash(Encoding.UTF8.GetBytes(pwd));
return Convert.ToBase64String(bytes);
```

Hash at submission — plaintext never stored or held in memory

EF Core async write

```
_db.UpgradeSchedules.Add(schedule);
await _db.SaveChangesAsync();
```

Releases web thread during DB round-trip; controller validates before Add()

Sequential chain

```
_db.ShellRequests.Add(request);
await _db.SaveChangesAsync();
await _emailService.SendTechOpsNotificationAsync(...);
```

Notify only after confirmed DB commit — no orphaned records

CSV export

```
csv.AppendLine("ID,Name,Version,Status,Date");
foreach (var s in schedules) csv.AppendLine(...);
return File(Encoding.UTF8.GetBytes(csv.ToString()),
"text/csv", "report.csv");
```

100% server-side — no JavaScript; browser download via FileContentResult

Testing Strategy & Defect Resolution

Three-level structured testing across Units 5–7

Integration testing
5 cases

Layer boundaries — controllers ↔ EF Core ↔ SQL Server ↔ SendGrid

Caught D-01 (hashing) and D-02 (EF Core mapping)

System testing
3 cases

End-to-end black-box workflows — login, schedules, shell requests, CSV, error redirects

Confirmed all modules compose correctly

Acceptance testing
2 cases

Operational criteria — TechOps email delivered; CSV opens in Excel

Confirmed portal meets its operational purpose

Defects Identified & Resolved

ID	Root Cause	Resolution
D-01	SHA-256 hashing implemented separately in two modules	Extracted single HashPassword() helper
D-02	EF Core property names ≠ SQL Server column names	Added HasColumnName() mappings in AppDbContext
D-03	Session middleware not registered → 2FA codes discarded	Added AddSession() + UseSession() to Program.cs

Results & Evaluation Metrics

Quantitative performance + qualitative usability assessment

1.2s

Login response time

Form → auth → dashboard

2.4s

CSV export time

Query → build → download

0

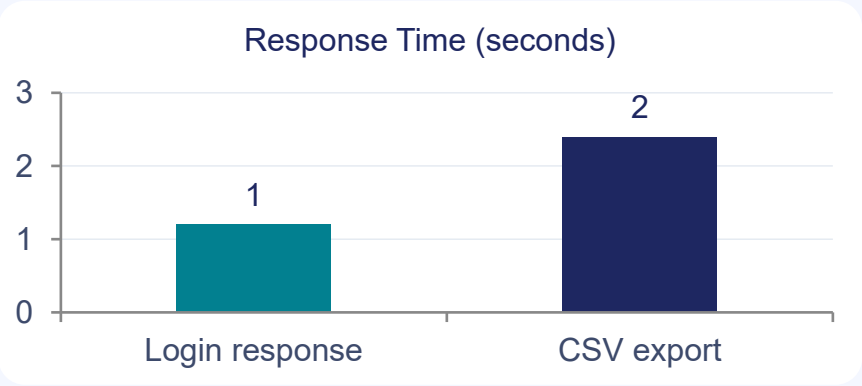
Build errors

dotnet build — clean

10/10

Test cases passed

All levels — no failures



Qualitative Assessment

- Navigation clarity

5 labelled modules — consistent orientation on every screen

- Action button consistency

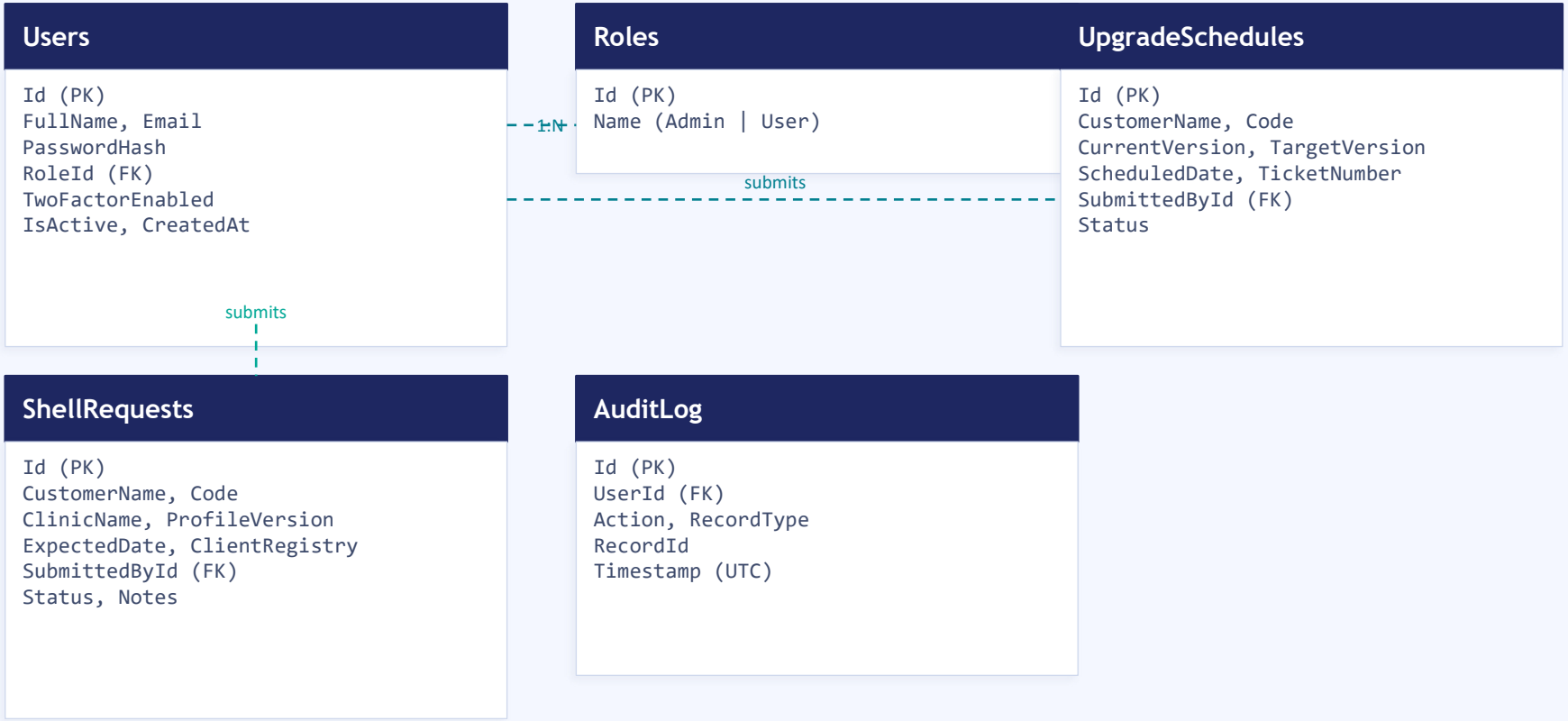
View / Complete / Cancel — same pattern on all list screens

- Dashboard visibility

4 summary cards — workload at a glance, no drilling required

Database Design — Entity Relationship

Five entities; EF Core migrations; explicit column mappings after D-02



Version Control & CI/CD Planning

GitHub two-branch strategy · semantic versioning · future pipeline

Repository structure

main (stable releases)

v1.0.0	v1.0.1	v1.0.2	v1.0.3	v1.0.4

development (active work)

Feature work → test → PR → merge to main

Each tag = rollback point + release notes (what changed, tested, known issues)

Configuration: credentials stored as environment variables — never committed to repo (Bennett, 2024)

CI/CD Pipeline (planned)

- GitHub Actions trigger

Every push to development branch

- dotnet restore /clean
- dotnet build
- dotnet test (xUnit — planned)
- Block merge to main if any step fails
- Auto release artifact on main

Digital Portfolio & Project Publication

GitHub Pages · GitHub Repository ·

GitHub Pages

`rauf600.github.io/UpgradePortal`

- Visual landing page with system overview
- Module screenshots — all 6 features
- Architecture & ERD diagrams embedded
- 8-section logical flow (overview → detail)

GitHub Repository

`github.com/Rauf600/UpgradePortal`

- Full source code — ASP.NET Core MVC project
- Structured README — setup, env vars, commands
- v1.0.0–v1.0.4 release tags with release notes
- main + development branch history
- Meaningful commit messages — full decision trail

LinkedIn portfolio entry planned — will link to both platforms and provide a professional project summary for industry reviewers.

Conclusion & Future Work

What was achieved

- Working prototype replacing fragmented spreadsheet coordination
- 10 / 10 test cases passed across 3 testing levels
- 3 defects identified and resolved before main branch
- 1.2s login · 2.4s export · zero build errors
- Published at rauf600.github.io/UpgradePortal

Next improvements (prioritised)

- Add [Authorize(Roles)] to all controller actions
- Create automated xUnit test suite
- Implement GitHub Actions CI/CD pipeline
- Complete Jira integration (v1.1.0) + analytics (v1.2.0)
- Structured load testing with JMeter / k6

Key lesson: Professional engineering discipline — MVC architecture, security-first design, semantic versioning, and staged testing — can be applied rigorously within a solo academic prototype. Every decision was deliberate; every defect was caught before it reached a user.

References

Alsubaei, F., Abuhussein, A., & Shiva, S. (2019). Security and privacy in the Internet of Medical Things. IEEE Access, 7, 489–507. https://doi.org/10.1109/ACCESS.2018.2886416

Bennett, L. (2024). Software configuration management in software engineering. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

BrowserStack. (2025). What is system testing? https://www.browserstack.com/guide/system-testing

GeeksforGeeks. (2025). Measuring software quality using quality metrics. https://www.geeksforgeeks.org/software-engineering/measuring-software-quality-using-quality-metrics/

Hrupa, A. (2024). What is system testing in software testing. Testfort. https://testfort.com/blog/what-is-system-testing

Kruse, C. S., et al. (2017). Cybersecurity in healthcare: A systematic review. Technology and Health Care, 25(1), 1–10. https://doi.org/10.3233/THC-161263

Laptick, S. (2022). How SDLC prototype model helps build comprehensive apps. XB Software. https://xbsoftware.com/blog/prototype-model-sdlc/

Narasimhan, A., Sham, R., & Subramanian, H. (2025). A beginner's handbook to DevOps [White paper]. Dell Technologies.

Red Hat. (2025). What is DevOps? https://www.redhat.com/en/topics/devops/what-is-devops

Summers, B. L. (2020). Effective methods for software engineering. Auerbach Publishers.

Walker, V. (2022). 14 software architecture design patterns to know. Red Hat. https://www.redhat.com/en/blog/14-software-architecture-patterns

Wells, J. (2025). Top 12 software quality metrics to measure and why. LinearB. https://linearb.io/blog/software-quality-metrics